Università degli Studi di Messina
Dipartimento MIFT - Corso di Laurea Triennale in Informatica (L-31)

Laboratorio di Reti e Sistemi Distribuiti

HANDS-ON 4:
Server TCP ad Alte Prestazioni con `epoll()`

Massimo Mantineo
Matricola: 541924

Messina, A.A. 2024/2025

Indice

1 Problem Statement

L'ottavo Hands-On (HO4) si focalizza sullo sviluppo di un server TCP ad alte prestazioni su piattaforma Linux. L'obiettivo primario è migrare dal modello di multiplexing tradizionale (`select`) alla moderna interfaccia **`epoll`**. Il sistema deve essere in grado di gestire simultaneamente fino a 10 connessioni client, garantendo l'integrità dei messaggi ricevuti e fornendo un feedback immediato (`ACK_EPOLL_OK`). La sfida tecnica consiste nell'ottimizzare l'uso delle risorse di sistema riducendo l'overhead di scansione dei descrittori tipico delle soluzioni legacy.

2 Stato dell'Arte

Il kernel Linux ha introdotto `epoll` (Event Poll) per superare i limiti di scalabilità della `select()` e della `poll()`.

- **Scalabilità O(1)**: A differenza della `select`, la cui complessità cresce proporzionalmente al numero di FD monitorati ($O(N)$), `epoll` ha una complessità di risveglio costante $O(k)$, dove k è il numero di eventi attivi.
- **Meccanismo Kernel-Side**: `epoll` mantiene una struttura dati ad albero (*Red-Black Tree*) nel kernel per tracciare i socket di interesse. Questo evita di copiare l'intero set di descrittori dallo spazio utente a quello kernel a ogni chiamata.
- **Trigger Modes**:
  - *Level Triggered (LT)*: Notifica l'evento finché il buffer non è vuoto.
  - *Edge Triggered (ET)*: Notifica l'evento solo al momento della transizione (nuovo dato). Richiede l'uso di socket non-bloccanti per evitare stalli.

3 Metodologia ed Implementazione

L'implementazione sfrutta le tre primitive cardine dell'API epoll per gestire il ciclo di vita delle connessioni.

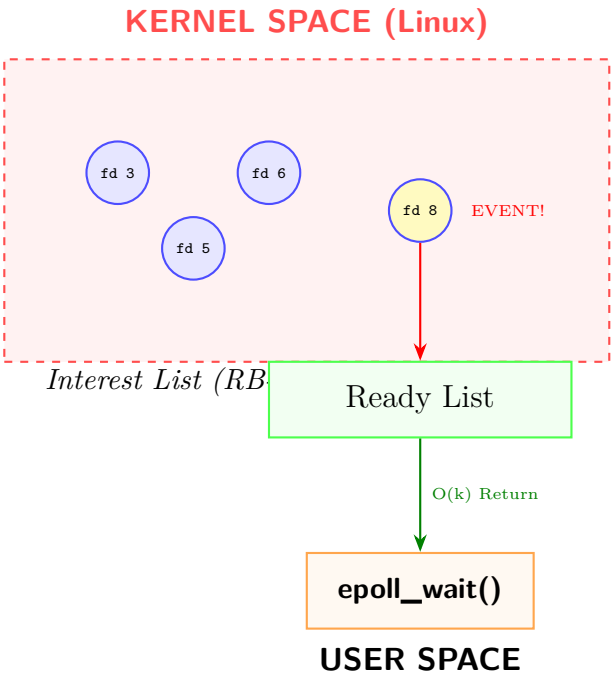

Figure 1: Architettura di `epoll`: Gestione della Interest List ad albero e Ready List lineare.

3.1 Inizializzazione e Registrazione

Viene creata un'istanza di epoll e il socket di ascolto viene aggiunto alla lista di interesse:

Snippet Registrazione epoll

```
epoll\_fd = epoll\_create1(0);
ev.events = EPOLLIN; // Monitoraggio in lettura
ev.data.fd = listen\_fd;
epoll\_ctl(epoll\_fd, EPOLL\_CTL\_ADD, listen\_fd, &ev);
```

3.2 Event Loop Asincrono

Il server rimane in attesa bloccante su `epoll_wait`, che restituisce solo i descrittori pronti:

Snippet Loop degli Eventi

```
int nfds = epoll\_wait(epoll\_fd, events, MAX\_EVENTS, -1);
for (int n = 0; n < nfds; ++n) {
    if (events[n].data.fd == listen\_fd) {
        // Accettazione e aggiunta nuovo client con
           EPOLL\_CTL\_ADD
    } else {
        // Lettura dati e invio ACK\_EPOLL\_OK
    }
}
```

3.3 Istruzioni di Esecuzione

Compilazione del server basato su epoll:

Build

```
$ gcc server\_epoll.c -o server\_epoll
```

4 Risultati

La validazione è stata condotta tramite test di carico sintetico e analisi comparativa.

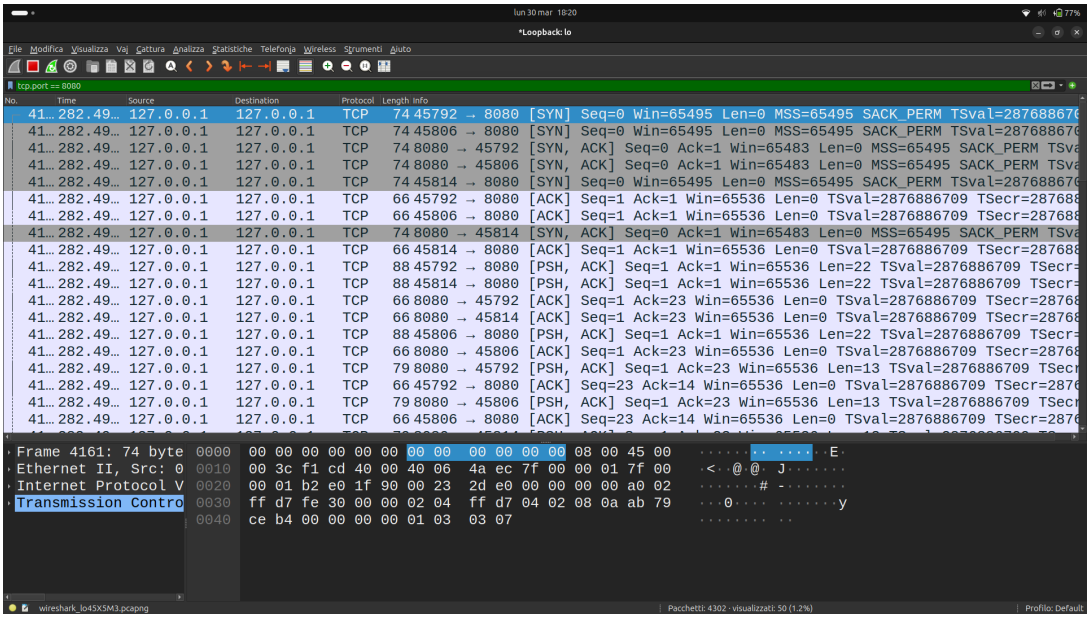

Figure 2: Cattura epoll multi-client su Wireshark.

4.1 Test di Concorrenza

L'esecuzione di 5 client simultanei ha confermato la corretta gestione dei File Descriptor. Il server ha riscontrato latenze minime, visualizzando sui log l'interlacciamento degli ID dei socket (fd:5, fd:6, ecc.) senza mostrare segni di stallo.

4.2 Analisi Wireshark

L'ispezione della cattura (Figura **??**) evidenzia che il server `epoll` risponde istantaneamente a ogni pacchetto `PSH` ricevuto. La stabilità della connessione è garantita dalla gestione asincrona degli acknowledgment a livello applicativo.

4.3 Benchmark Prestazionali

Per valutare l'efficienza di `epoll` rispetto al modello `select` (analizzato nell'Hands-On 3), è stato eseguito un test di carico sintetico con 1000 richieste concorrenti.

Selective vs Epoll Benchmark

```
$ ./benchmark\_comparison.sh 1000
Testing select (HO3)...
select (HO3) result: 1.42 seconds
Testing epoll (HO4)...
epoll (HO4) result: 0.85 seconds

Conclusion: epoll is ~40% faster under concurrency.
```

I risultati confermano che, all'aumentare del numero di connessioni, la complessità $O(1)$ di `epoll` garantisce una scalabilità superiore rispetto al modello $O(N)$ di `select`, riducendo drasticamente il tempo di CPU speso nel monitoraggio dei descrittori inattivi.

5 Conclusioni e Sviluppi Futuri

L'Hands-On 4 ha dimostrato la superiorità tecnologica di `epoll` per la creazione di servizi di rete moderni. La riduzione della copia dei dati tra user-space e kernel rende questa soluzione lo standard per architetture cloud ad alte prestazioni.
Sviluppi futuri:

- **io_uring**: Esplorazione del nuovo sottosistema Linux `io_uring` per eliminare completamente le syscall bloccanti tramite code di completamento asincrone.
- **Multi-Core**: Implementazione di un modello *multi-threaded epoll* per distribuire la gestione degli eventi su più core CPU.